REPÚBLICA PORTUGUESA
EDUCAÇÃO, CIÊNCIA E INOVAÇÃO

AEGP
AGRUPAMENTO DE ESCOLAS GABRIEL PEREIRA - ÉVORA

AGRUPAMENTO DE ESCOLAS GABRIEL PEREIRA - ÉVORA

# Programação e Sistemas de Informação

## MÓDULO 17A

## TÉCNICAS DE ACESSO A BASES DE DADOS

CURSO PROFISSIONAL
TÉCNICO DE GESTÃO E PROGRAMAÇÃO DE SISTEMAS INFORMÁTICOS
PROGRAMAÇÃO E SISTEMAS DE INFORMAÇÃO

# Técnicas de Acesso a Bases de Dados



MYSQL SERVER

SQL SERVER

LINQ

# Técnicas de Acesso a Bases de Dados

- A utilização de base de dados é indispensável na criação de aplicações, mais complexas, que permitam criar, alterar e consultar informação;

- É necessário desenvolver competências que permitam:
  - Conhecer e aplicar diferentes mecanismos de acesso a base de dados;
  - Utilizar componentes específicos de acesso a base de dados;
  - Construir uma interface de acesso a uma base de dados;
  - Inserir, apagar, atualizar e procurar informação numa base de dados;
  - Utilizar técnicas avançadas de consultas numa base de dados.

# Técnicas de Acesso a Bases de Dados



- MySQL Server
  - Utiliza o MySQL Workbench para criar e alojar a base de dados.
- SQL Server
  - Utiliza o SQL Server Compact Edition para criar a base de dados.
- LINQ
  - Utiliza bibliotecas e mecanismos específicos do Visual Studio para criar e utilizar a base de dados.

# MYSQL SERVER

## (*MYSQL WORKBENCH*)

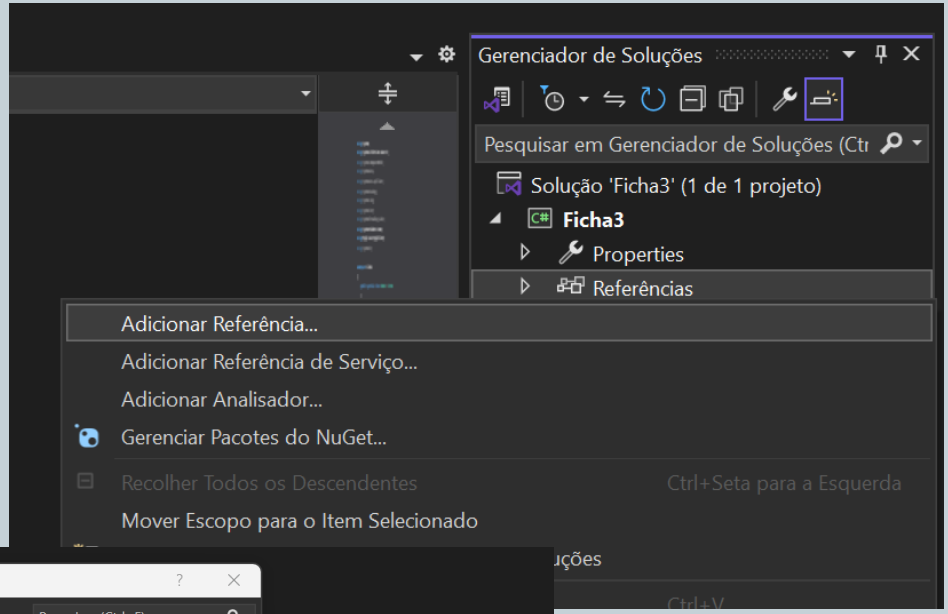
# MySql Workbench - Visual Studio

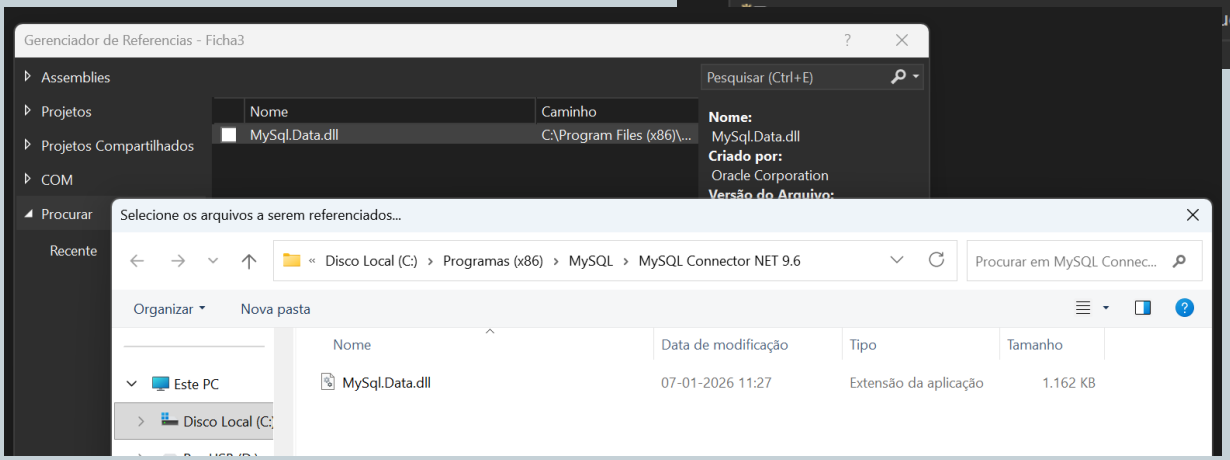
- Acesso a bases de dados criada no MySql Workbench
  - Criar a base de dados no MySql Workbench
    - Diretamente no servidor (localhost);
    - Criar esquema físico da tabela num modelo e utilizara a opção de "Forward Enginneering" no menu "Database" para criar a base de dados (módulos 14 e 15);
  - Ligação do C# à base de dados
    - Instalar o conector disponível em:

      https://dev.mysql.com/downloads/connector/net/

# MySql Workbench - Visual Studio

- Adicionar uma referência à solução (tecla direita):
- Na janela seguinte, clicar em procurar e adicionar o ficheiro MySqlData.dll no caminho indicado:



# MySql Workbench - Visual Studio

- Verificar a existências destes dois *namespace*

```
using MySql.Data.MySqlClient;
using System.IO;
```

- Criar variáveis para a conexão e para a informação relativa à conexão

```
MySqlConnection conexao;
const string infConexao = "datasource=localhost;port=3306;username=root;password=mysql;database=inscricoes";
```

# MySql Workbench - Visual Studio

- Exemplo de conexão a uma base de dados na conexão referida e carregando todos os registos da tabela para uma lista no método PreencheLista()

```
private void Form1_Load(object sender, EventArgs e)
{
    try
    {
        conexao = new MySqlConnection(infConexao);
        conexao.Open();

        //Verificação da conexão
        MessageBox.Show("Conexão efetuada com sucesso");

        PreencheLista();

        //Incialização do campo Camisola na abertura do Form
        numcamisola = atletas.Count + 1;
        lblCamisola.Text = numcamisola.ToString();

    }
    catch (Exception ex)
    {
        MessageBox.Show(Form1.ActiveForm, ex.Message, "Erro", MessageBoxButtons.OK, MessageBoxIcon.Error);

    }
}
```

# MySql Workbench - Visual Studio

- Método_PreencheLista()
  - Percorre todos os registos obtidos através da instrução de SQL e adiciona-os à lista *atletas;* utiliza uma variável auxiliar *a (struct atleta)*

```
private void PreencheLista()
{
    string sql = "select * from atleta";
    using (MySqlCommand cmd = new MySqlCommand(sql, conexao))
    {
        using (MySqlDataReader reader = cmd.ExecuteReader())
        {
            while (reader.Read())
            {
                a.camisola = reader.GetInt32("numCamisola");
                a.nome = reader.GetString("nome");
                a.clube = reader.GetString("clube");
                a.prova = reader.GetString("prova");
                atletas.Add(a);
            }
        }
    }
}
```

# MySql Workbench - Visual Studio

- Adicionar um registo à base de dados
  - Possível código para inserir um registo numa tabela da base de dados;
  - O registo é adicionado à lista *atletas*, e em seguida inserido na base de dados.

```
private void BtInserir_Click(object sender, EventArgs e)
{
    //Verificar se todos os campos estão preenchidos
    if (txtNome.Text!="" && cbxClube.Text !="" && cbxProva.Text !="")
    {
        //Guardar novo registo
        a.camisola = int.Parse(lblCamisola.Text);
        a.nome = txtNome.Text;
        a.clube = cbxClube.Text;
        a.prova = cbxProva.Text;

        atletas.Add(a);

        //Insere o registo na BD
        string sql = "insert into atleta values(@camisola,@nome,@clube,@prova)";
        using (MySqlCommand cmd = new MySqlCommand(sql, conexao))
        {
            cmd.Parameters.AddWithValue("@camisola", a.camisola);
            cmd.Parameters.AddWithValue("@nome", a.nome);
            cmd.Parameters.AddWithValue("@clube", a.clube);
            cmd.Parameters.AddWithValue("@prova", a.prova);

            cmd.ExecuteNonQuery();

            MessageBox.Show("Registo inserido!", "Informação");
        }

        //Preparar os campos para inserir outro registo
        lblCamisola.Text = (atletas.Count + 1).ToString();
        txtNome.ResetText();
        cbxClube.ResetText();
        cbxProva.ResetText();
        txtNome.Focus();

    }
    else
    {
        //Mensagem de que os campos devem estar todos preenchidos
        MessageBox.Show("Preencha todos os campos", this.Text,
            MessageBoxButtons.OK, MessageBoxIcon.Information);
    }
}
```

# Técnicas de Acesso a Bases de Dados



## SQL SERVER

(*MICROSOFT*)

# SQL Server - Visual Studio

- System.Data.SqlServerCe
  - Namespace da plataforma .NET que permite trabalhar com o SQL Server Compact Edition, uma versão leve e incorporada do SQL Server;
  - Fornece classes para aceder e manipular bases de dados.
  - Contém classes como:
    - **SqlCeConnection** - estabelece ligação à base de dados
    - **SqlCeEngine** - permite criar e gerir a base de dados
    - **SqlCeCommand** - executa comandos SQL na base de dados
    - **SqlCeDataReader** - lê dados de forma sequencial e rápida
    - **SqlCeDataAdapter** - intermediário entre base de dados e DataTable/DataSet
    - **DataTable** - representa uma tabela em memória

# SQL Server - Visual Studio



- Criar base de dados
  - SqlCeEngine
    - Criação do ficheiro .sdf.
    - Compactação e manutenção da base de dados

```
SqlCeEngine <nome_objecto> = new SqlCeEngine();
SqlCeEngine(), pode ter duas assinaturas:
1.()
Obriga a ter uma ligacão numa linha seguinte
<nome_objecto>.LocalConnectionString = <caminho>;

2.(string connectionString)

<nome_objecto>.CreateDatabase();
```

# SQL Server - Visual Studio

- Ligar a uma base de dados
  - SqlCeConnection
    - Utiliza uma connection string.
    - Permite abrir e fechar a ligação

```
SqlCeConnection <nome_objecto> = new SqlCeConnection();

SqlCeConnection(), pode ter duas assinaturas
1.()
Obriga a ter uma ligacão numa linha seguinte
<nome_objecto>.ConnectionString = <caminho>;

2.(string connectionString)
<nome_objecto>.Open();
```

# SQL Server - Visual Studio

- Criar tabela na base de dados
  - 1º - Ligar à base de dados

```
SqlCeConnection <nome_objecto> = new SqlCeConnection();
<nome_objecto>.ConnectionString = <caminho>;
<nome_objecto>.Open();
```

  - 2º - Criar a tabela

```
SqlCeCommand <nome_objecto> = new SqlCeCommand();
SqlCeCommand(), pode ter três assinaturas:
1.()
Obriga a ter uma ligacão numa linha seguinte
<nome_objecto>.CommandText = <comando SQL>;

2.(string commandText)

3.(string commandText, SqlCeConnection connection)
<nome_objecto>.ExecuteNonQuery();
<nome_objecto>.Dispose();
```

**SqlCeCommand:**

- ✓ Pode executar queries e comandos não-query.
- ✓ Métodos comuns: ExecuteReader(), ExecuteNonQuery().

# SQL Server - Visual Studio



- Criar tabela para receber dados
  - Data Table
    - Armazena dados temporariamente.
    - Pode ser preenchida com SqlCeDataAdapter.
    - Permite manipular dados sem ligação ativa.
  - SqlCeDataAdapter
    - Permite preencher estruturas de dados em memória.
    - Facilita operações desconectadas

```
DataTable <nome_objecto> = new DataTable();

SqlCeDataAdapter <nome_objecto1> = new SqlCeDataAdapter(string selectCommandText,
SqlCeConnection selectConnection);

<nome_objecto1>.Fill(<nome_objecto>);

<ligacao>.Dispose();
<nome_objecto1>.Dispose();
```

# SQL Server - Visual Studio



- Mostrar os Dados

```
var <variável> = <objecto>.Rows[<indice da linha>];
```

  - Numa label

```
label.Text = <variável>["<campo da base de dados>"].ToString();
```

  - Numa Listbox

```
foreach (DataRow item in <objecto>.Rows)
  {
   listBox1.Items.Add(item["<campo da base de dados>"]+" -> "+
item["<campo da base de dados>"]+" -> "+item["<campo da base de
dados>"]);
  }
```

  - Numa Listbox

```
dataGridView1.DataSource = dados;
dataGridView1.Columns["<campo da base de dados>"].Visible = false;
dataGridView1.Columns["<campo da base de dados>"].Width = 200;
```

# SQL Server - Visual Studio



- Inserir Dados

```
SqlCeCommand <objecto> = new SqlCeCommand (string commandText,
SqlCeConnection connection);

<objecto>.ExecuteNonQuery();
```

Exemplo:

```
string query = "INSERT INTO t_filmes VALUES('" + txtNome.Text + "','" +
txtTelefone.Text + "','" + txtMorada.Text + "')";
```

# SQL Server - Visual Studio



- Atualizar Dados

```
SqlCeCommand <objecto> = new SqlCeCommand(string commandText,
SqlCeConnection connection);

<objecto>.ExecuteNonQuery();
```

Exemplo:

```
string query = "UPDATE pessoas set nome='"+txtNome.Text+"',
morada='"+txtMorada.Text+"',telefone='"+ txtTelefone.Text+"' where
nome='"+txtAtualizar.Text +"'";
```

# SQL Server - Visual Studio



- Eliminar Dados

```
SqlCeCommand <objecto> = new SqlCeCommand(string commandText,
SqlCeConnection connection);

<objecto>.ExecuteNonQuery();
```

Exemplo:

```
string query = "DELETE t_filmes WHERE nome_filme='"+txtNome.Text+"'";
```

# Técnicas de Acesso a Bases de Dados

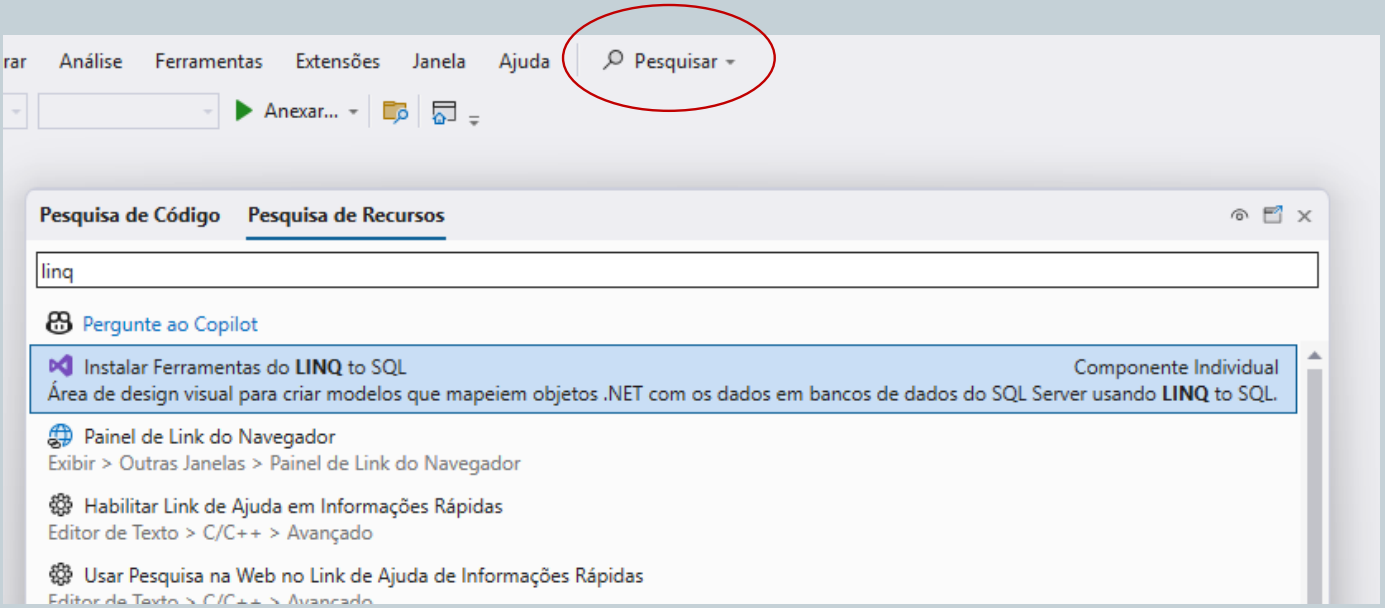


## LINQ

(*LANGUAGE INTEGRATED QUERY*)

# LINQ - Visual Studio

- Acesso a bases de dados através da **LINQ** (*Language INtegrated Query*).
- Para isso, é necessário instalar o componente ***Ferramentas do LINQ to SQL:***

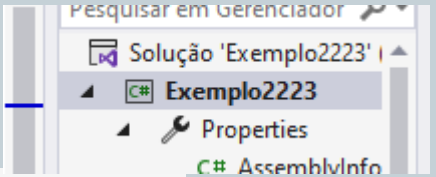


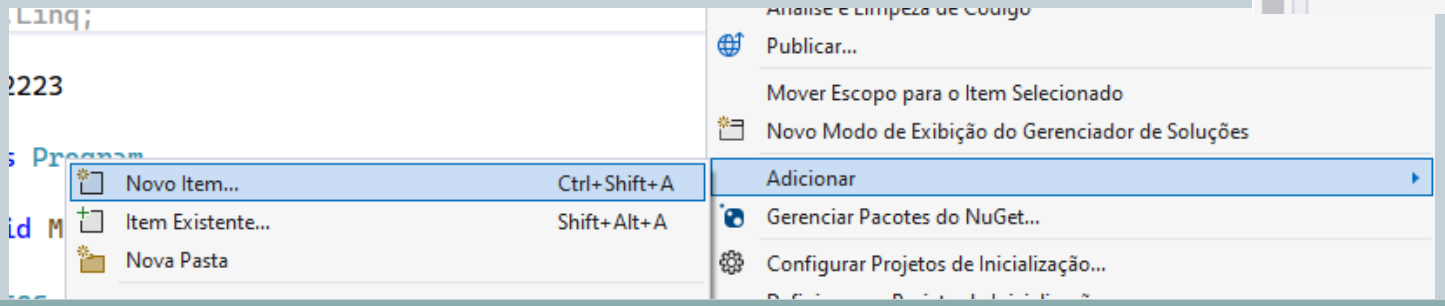
# LINQ - Visual Studio

- No código, utiliza-se a classe **DataContext**, definidas na ***namespace*** **System.Data.Linq;**

https://docs.microsoft.com/en-us/dotnet/api/system.data.linq.datacontext?view=netframework-4.8

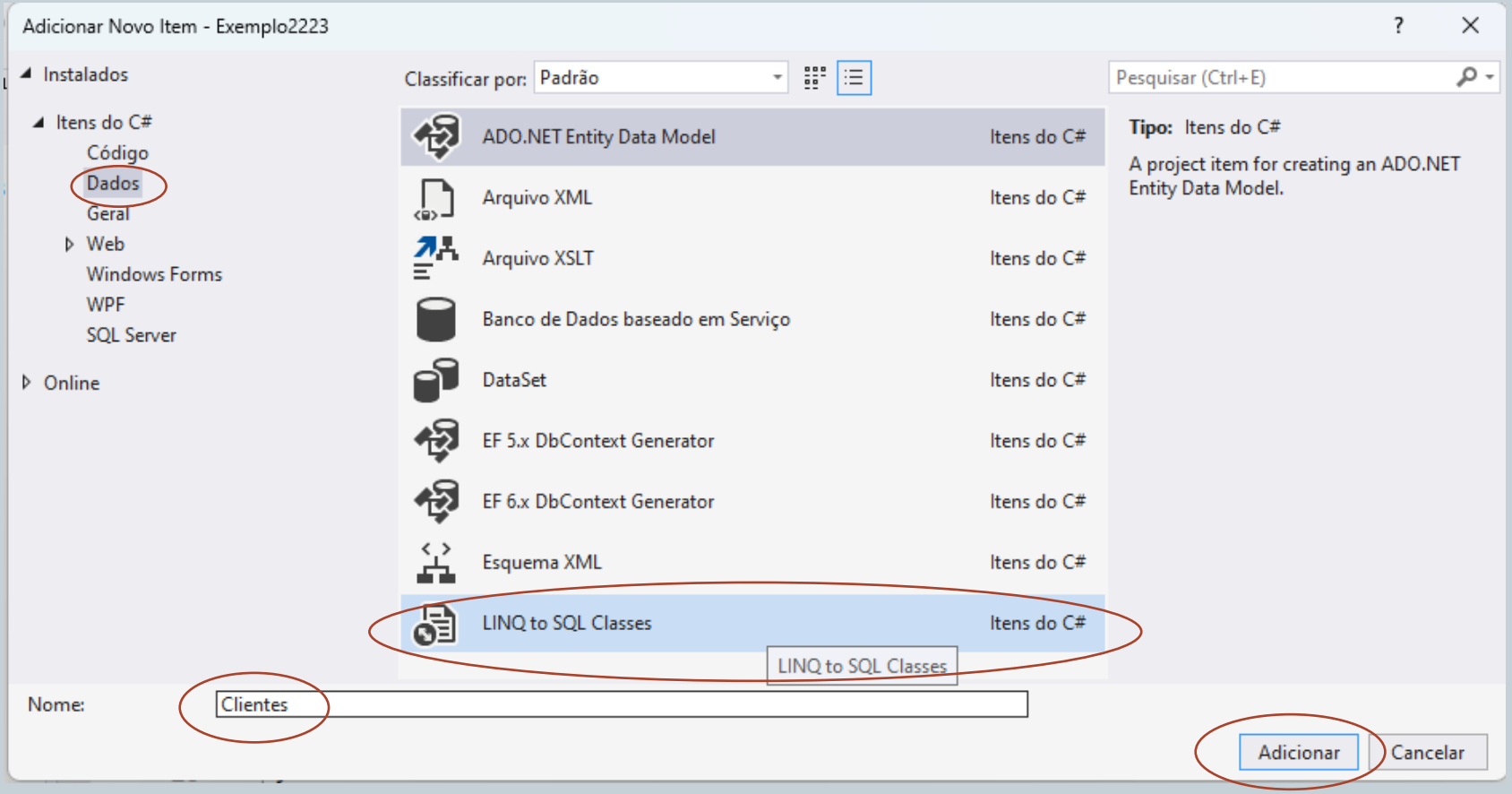
- É necessário incluir ***using System.Data.Linq***.
- Criação da Classe DataContext
  - Em Solution Explorer, clicar com o botão direito do rato (menu de contexto) no projeto em que está a trabalhar
  - e escolher Add – New Item



# LINQ - Visual Studio

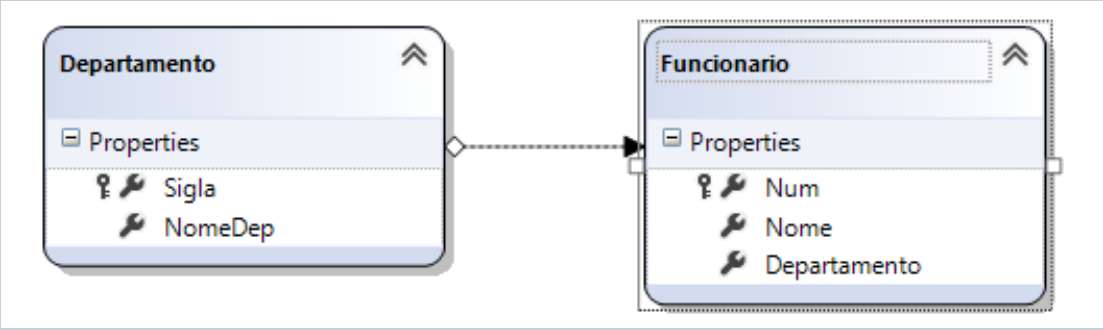
- Criação da Classe DataContext



# LINQ - Visual Studio

- Exemplo de uma base de dados

```
CREATE TABLE [dbo].[Departamentos] (
    [Sigla]   VARCHAR (10) NOT NULL,
    [NomeDep] VARCHAR (50) NOT NULL,
    PRIMARY KEY CLUSTERED ([Sigla] ASC)
);
CREATE TABLE [dbo].[Funcionarios] (
    [Num]          INT          IDENTITY (1, 1) NOT NULL,
    [Nome]         VARCHAR (50) NOT NULL,
    [Departamento] VARCHAR (10) NOT NULL,
    PRIMARY KEY CLUSTERED ([Num] ASC),
    CONSTRAINT [Rel_Fun_Dep] FOREIGN KEY ([Departamento]) REFERENCES
[dbo].[Departamentos] ([Sigla])
);
```



# LINQ - Visual Studio



- Consultar informação

```
//Declaração da variável dc que é um novo objecto DataContext
DataClassesNome dc = new DataClassesNome;

//Criação de uma lista (var=sem tipo de dados definido) de registos da tabela indicada
var lista = from NomeTabela in dc.NomeTabela select NomeTabela;

//Utilização de um ciclo foreach para listar todos os elementos da lista
```

- Consultar informação com ordenação

```
//Criação da lista ordenada
var lista = from [NomeTabela] in dc.[NomeTabela] orderby [NomeTabela].[NomeCampo]
select [NomeTabela];
```

- Contar registos

```
lista.Count();
```

# LINQ - Visual Studio



- Exemplo de consulta de informação / ordenação /contar

```
//Criação do objeto Data Context
EmpresaDataContext dc = new EmpresaDataContext();

//Criação de uma lista com todos os funcionários
var lista = from Funcionarios in dc.Funcionarios
            orderby Funcionarios.Nome descending
            select Funcionarios;
foreach (Funcionario func in lista)
{
    Console.WriteLine("Número: " + func.Num);
    Console.WriteLine("Nome: " + func.Nome);
    Console.WriteLine("Departamento: " + func.Departamento);
}

//Mostrar o número de funcionários
Console.WriteLine("Número de Funcionários: " + lista.Count());
```

# LINQ - Visual Studio



- Pesquisa de informação

```
//Criação da lista com critérios
var lista = from NomeTabela in dc.NomeTabela where
NomeTabela.NomeCampo == critério select NomeTabela;

//Texto começado por xxx
var lista = from NomeTabela in dc.NomeTabela where
NomeTabela.NomeCampo.StartsWith("xxx") select NomeTabela;

//Texto contém xxx
var lista = from NomeTabela in dc.NomeTabela where
NomeTabela.NomeCampo.Contains("xxx") select NomeTabela;
```

# LINQ - Visual Studio

- Exemplo de pesquisa

```
//Mostrar os funcionários com número de funcionário = 2
var listanum = from Funcionarios in dc.Funcionarios
               where Funcionarios.Num == 2
               select Funcionarios;
foreach (Funcionario func in listanum)
{
    Console.WriteLine("Número: " + func.Num);
    Console.WriteLine("Nome: " + func.Nome);
    Console.WriteLine("Departamento: " + func.Departamento);
}
Console.ReadLine();

//Mostrar os funcionários cujo nome inicia com a letra A
var listanome = from Funcionarios in dc.Funcionarios
                where Funcionarios.Nome.StartsWith("A")
                select Funcionarios;
foreach (Funcionario func in listanome)
{
    Console.WriteLine("Número: " + func.Num);
    Console.WriteLine("Nome: " + func.Nome);
    Console.WriteLine("Departamento: " + func.Departamento);
}
```

# LINQ - Visual Studio



- Junção entre tabelas

```
var lista = from NomeTabela1 in dc.NomeTabela1 join NomeTabela2 in


dc.NomeTabela2 on NomeTabela1.NomeCampo1 equals
                                        NomeTabela2.NomeCampo2
select new
{
        NomeTabela.Campo1, ..., NomeTabela.Campon
}
```

# LINQ - Visual Studio



- Exemplo de junção entre tabelas

```
var listajoin = from Funcionarios in dc.Funcionarios
                join Departamentos in dc.Departamentos
                on Funcionarios.Departamento equals
                                        Departamentos.Sigla
                select new
                {
                    Funcionarios.Num,
                    Funcionarios.Nome,
                    Departamentos.NomeDep
                };

   foreach (var c in listajoin)
   {
       Console.WriteLine("Número: " + c.Num);
       Console.WriteLine("Nome: " + c.Nome);
       Console.WriteLine("Departamento: " + c.NomeDep);
   }
```

# LINQ - Visual Studio



- Inserção de registos

```
//Criar um novo objeto da classe onde queremos inserir o registo
NomeClasse NomeVariável = new NomeClasse();

//Atribuir informação a cada campo
NomeVariável.NomeCampo1 = xxx;
NomeVariável.NomeCampo2 = yyy;
...
NomeVariável.NomeCampon = zzz;

//Gravar a informação
dc.NomeClasse.InsertOnSubmit(NomeVariável);
dc.SubmitChanges();
```

# LINQ - Visual Studio



- Exemplo de inserção de registo

```
Funcionario func2 = new Funcionario();
func2.Num = 30;
func2.Nome = "Carlos Santos";
func2.Departamento = "FIN";

dc.Funcionarios.InsertOnSubmit(func2);
dc.SubmitChanges();
```

# LINQ - Visual Studio



- Alteração de registos

```
//Criar um novo objecto da classe onde queremos alterar o registo
NomeClasse NomeVariável = new NomeClasse();

//Pesquisar o registo que vai ser alterado
var pesquisa = from NomeTabela in dc.NomeTabela where NomeTabela.NomeCampo == critério select
NomeTabela;

//Seleção de um único registo
NomeVariável = pesquisa.Single();
NomeVariável.NomeCampo = xxx;

//Gravar a informação
dc.SubmitChanges();
```

# LINQ - Visual Studio

- Exemplo de alteração de registos

```
Funcionario func2 = new Funcionario();

    var pesquisa = from Funcionarios in dc.Funcionarios
                   where Funcionarios.Num == 30 select
                                 Funcionarios;

    func2 = pesquisa.Single();
    func2.Nome = "Carlos Alberto Santos";
    func2.Departamento = "COM";

    dc.SubmitChanges();
```

# LINQ - Visual Studio

- Eliminação de registos

```
//Criar um novo objecto da classe onde queremos eliminar o registo
NomeClasse NomeVariável = new NomeClasse();

//Pesquisar o registo que vai ser eliminado
var pesquisa = from NomeTabela in dc.NomeTabela where NomeTabela.NomeCampo == critério select NomeTabela;

//Seleção de um único registo
NomeVariável = pesquisa.Single();
dc.NomeTabela.DeleteOnSubmit(NomeVariável);

//Gravar a informação
dc.SubmitChanges();
```

# LINQ - Visual Studio



- Exemplo de eliminação de registos

```
Funcionario func2 = new Funcionario();

    var pesquisa = from Funcionarios in dc.Funcionarios
                   where Funcionarios.Num == 30 select
                                        Funcionarios;

    func2 = pesquisa.Single();
    dc.Funcionarios.DeleteOnSubmit(func2);

    dc.SubmitChanges();
```